

AI-DLC v2 Power — Example Guide

A worked walkthrough of using the AIDLC v2 Kiro Power

AWS Samples — sample-aidlc-v2-kiro-power

September 5, 2026

Contents

What Is the AI-DLC v2 Power?	3
Prerequisites	3
Step 1 — Install the Power	3
Option A — Import from GitHub	3
Option B — Import from a Local Folder	3
What the Power Installs	4
Step 2 — Start an AI-DLC Workflow	4
Step 3 — How the Workflow Operates	4
Inception Phase (What & Why)	4
Construction Phase (How)	4
Adaptive Behavior	5
Step 4 — Answering Structured Questions	5
A Complete Worked Example	5
The Goal	5
1. State Your Intent	5
2. Intent Bootstrap	6
3. Reverse Engineering (Existing Code)	6
4. Requirements & Functional Design	6
5. Code Generation	6
6. Build & Test	6
7. Review the Artifacts	6
Advanced Usage	7
Reverse Engineering Existing Codebases	7
Custom Workflow Composition	7
Key Principles	7
Tips	7

Troubleshooting	8
Important Notes	8

What Is the AI-DLC v2 Power?

AI-DLC (AI-Driven Life Cycle) is an autonomous, structured software-development workflow. The **AI-DLC v2 Kiro Power** installs this workflow into your Kiro workspace so you can build software through deterministic, reviewable phases — from capturing intent, through requirements and design, to verified code generation.

Once installed, the power adds an `.kiro/` directory to your project containing agents, skills, shared conventions, and a prompt hook. At the start of each new conversation, it asks once whether you want to use the AI-DLC workflow or proceed with normal Kiro behavior. Your choice applies for the whole session.

Key idea: you stay in the loop. AI-DLC produces an artifact at each stage, you review and approve it, and only then does the next stage run.

Prerequisites

Before using the power, make sure you have:

- **Kiro IDE** installed and running.
- **Node.js** on your **PATH** (used by the build script and the deterministic process checker).
- **Internet access** (the power downloads the AIDLC v2 branch from GitHub).
- On macOS/Linux: `curl` and `unzip` (pre-installed on most systems).

Use Kiro in **Vibe mode** for AI-DLC workflows. If nudged to switch to Spec mode, decline.

Step 1 — Install the Power

You can add the power from the Kiro **Powers** tab in two ways.

Option A — Import from GitHub

1. Open Kiro IDE.
2. Go to the **Powers** tab.
3. Click **Add Custom Power**.
4. Select **Import from GitHub**.
5. Paste the repository URL:

`https://github.com/aws-samples/sample-aidlc-v2-kiro-power`

Option B — Import from a Local Folder

1. Download the repository as a ZIP from GitHub and unzip it.
2. Open Kiro IDE and go to the **Powers** tab.
3. Click **Add Custom Power**.
4. Select **Import from Local Folder**.
5. Browse to the extracted folder and select the `aidlc-v2/` directory.

What the Power Installs

When activated, the power downloads the v2 branch of `awslabs/aidlc-workflows`, builds the Kiro distribution with `make build-kiro`, and copies the output into your workspace:

```
<project-root>/
`-- .kiro/
    |-- agents/          # AI-DLC builder and validator agents
    |-- aidlc-common/    # Shared conventions, protocols, and scripts
    |   |-- conventions/ # Folder structure, question format, state schema
    |   |-- protocols/   # Builder, orchestrator, and validator protocols
    |   `-- scripts/     # Deterministic process checker (Node.js)
    |-- hooks/           # Process-check + prompt-choice hooks
    `-- skills/          # AI-DLC skill set (orchestrator, requirements, ...)
```

After installation, **restart your Kiro session** so the new skills and hooks load.

Step 2 — Start an AI-DLC Workflow

Begin any prompt with “**Using AI-DLC, ...**” to activate the orchestrator directly. For example:

- “Using AI-DLC, build a REST API for user management”
- “Using AI-DLC, create a React dashboard for analytics”
- “Using AI-DLC, design a microservices architecture for an e-commerce platform”
- “Using AI-DLC, add authentication to my Express app”

If you do not prefix your message, at the start of a new conversation the power asks once:

“Would you like to use **AI-DLC (recommended)** for this session, or proceed normally?”

Choose **AI-DLC** for new features, major changes, or structured development. Choose **normal** for quick fixes, questions, or simple edits.

Step 3 — How the Workflow Operates

The orchestrator activates automatically and proposes a workflow tailored to your intent. The typical flow spans two phases.

Inception Phase (What & Why)

1. **Intent Bootstrap** — captures your development intent and asks structured questions.
2. **Requirements Analysis** — generates formal requirements from your answers.
3. **User Stories** — creates user stories (conditional; complex user-facing features).

Construction Phase (How)

4. **Functional Design** — technology-agnostic business logic artifacts.

5. **Application Design** — architecture and component design.
6. **Infrastructure Design** — infrastructure considerations.
7. **NFR Assessment** — non-functional requirements evaluation.
8. **NFR Design** — NFR patterns and logical components.
9. **Code Generation** — deterministic code generation with verification.
10. **Unit Generation** — test generation for the produced code.

Adaptive Behavior

The workflow adapts to complexity — only stages that add value run:

- **Simple bug fix:** Intent Bootstrap → Requirements (minimal) → Code Generation → Build & Test
 - **New feature in existing app:** Intent Bootstrap → Reverse Engineering → Requirements → Functional Design → Code Generation → Build & Test
 - **Greenfield application:** Intent Bootstrap → Requirements (comprehensive) → User Stories → Application Design → Units Generation → [Per-unit Construction] → Build & Test
-

Step 4 — Answering Structured Questions

AI-DLC gathers requirements through multiple-choice questions. Reply with the `[Answer]:` tag followed by your choice:

Which approach do you prefer for user authentication?

- A) OAuth 2.0 with third-party providers (Google, GitHub)
- B) Traditional username/password with JWT tokens
- C) Multi-factor authentication with email verification
- D) Social login only (no traditional signup)
- E) Other (please describe)

`[Answer]: B`

A Complete Worked Example

This example walks through building a small feature end to end using AI-DLC.

The Goal

Add token-based authentication to an existing Express API.

1. State Your Intent

Using AI-DLC, add JWT authentication to my Express app

The orchestrator recognizes an existing codebase and routes to the “**new feature in existing app**” flow.

2. Intent Bootstrap

AI-DLC asks a few structured questions to scope the work, for example:

What should protected routes do when a token is missing or invalid?

- A) Return 401 Unauthorized with a JSON error body
- B) Redirect to a login page
- C) Return 403 Forbidden
- D) Other (please describe)

[Answer]: A

Where should the token secret come from?

- A) Environment variable (process.env.JWT_SECRET)
- B) A config file committed to the repo
- C) A secrets manager (AWS Secrets Manager)
- D) Other (please describe)

[Answer]: A

3. Reverse Engineering (Existing Code)

Because this is an existing app, AI-DLC analyzes the current routes and middleware structure so its changes fit the established patterns. You can also trigger this explicitly:

Using AI-DLC, reverse-engineer the authentication module

4. Requirements & Functional Design

AI-DLC produces requirements and a functional design describing the login flow, token issuance, and route protection — all as reviewable artifacts. You approve each before moving on.

5. Code Generation

With designs approved, the builder agent generates the middleware, the login route, and the wiring into your app. The validator agent checks the output against the requirements and design.

6. Build & Test

The workflow builds and tests the change. Unit generation adds tests for the new middleware and login handler.

7. Review the Artifacts

Generated documentation lands next to your project:

```
org-ai-kb/aidlc-docs/intent-<nnn>-<slug>/
|-- aidlc-state.md           # Workflow state tracking
|-- audit.md                 # Complete audit trail
|-- inception/               # Planning phase artifacts
```

```

|   |-- requirements/
|   |-- application-design/
|   `-- plans/
`-- construction/          # Implementation phase artifacts
    |-- [unit-name]/
    |   |-- functional-design/
    |   |-- nfr-requirements/
    |   |-- nfr-design/
    |   `-- code/
    `-- plans/

```

Generated **source code** goes into your project directory; documentation artifacts stay in the `org-ai-kb/aidlc-docs/` tree.

Advanced Usage

Reverse Engineering Existing Codebases

Using AI-DLC, reverse-engineer the authentication module

This analyzes and documents existing code before any changes are made.

Custom Workflow Composition

Using AI-DLC, compose a workflow that skips wireframes and goes straight to code generation

The workflow-composition skill lets you tailor the pipeline to your needs.

Key Principles

- **Human-in-the-loop** — you review and approve each artifact before the next stage runs.
 - **Deterministic verification** — the process checker validates outputs at each transition.
 - **Artifacts are external** — generated docs land in `org-ai-kb/aidlc-docs/intent-<nnn>-<slug>/`.
 - **Code stays in the project** — generated source goes into your project directory.
 - **Adaptive execution** — only stages that add value run.
-

Tips

- Use Kiro in **Vibe mode**; decline if nudged toward Spec mode.
- Be specific in your intent statement for better workflow routing.
- Review each stage's output carefully before approving.
- The orchestrator may skip stages that are not relevant.
- You can ask to re-run a stage with different parameters.
- Trust the adaptive assessment — simple changes skip most conditional stages.

Troubleshooting

Problem	Solution
Download fails	Ensure internet access. Verify <code>curl</code> (macOS/Linux). Remove any existing <code>aidlc-workflows-v2/</code> folder and retry.
<code>make build-kiro</code> fails	Ensure Node.js is installed and on PATH. Try <code>make clean && make build-kiro</code> .
<code>make</code> not found (Windows)	Use Git Bash to run <code>bash targets/kiro/build.sh</code> , or install via <code>choco install make</code> .
Skills not loading	Confirm <code>.kiro/</code> exists at the project root with <code>agents/</code> , <code>aidlc-common/</code> , <code>hooks/</code> , <code>skills/</code> . Restart Kiro.
Permission denied	Check workspace write permissions. On macOS/Linux try <code>chmod -R u+w .kiro/</code> .

Important Notes

- v2 is **pre-release** and actively evolving — expect breaking changes.
- Generative AI can make mistakes — review all output.
- Licensed under **MIT-0**.

This guide summarizes the AI-DLC v2 Kiro Power. For the source workflow, see [aws-labs/aidlc-workflows](#) (v2 branch) on GitHub.